Concrete Project Proposal - Group 40

System Description

The system selected for this project is P.I.T.C.H., a microserivce-based AI-assisted web application developed as part of our capstone project. The system includes:

- Next.js frontend
- NestJS backend microservices
- API Gateway
- RabbitMQ for asynchronous workflows
- Redis for caching
- Persistent data storage
- Docker-based deployment

Services communicate via HTTP APIs and synchronous messaging. Each service is independently deployable and horizontally scalable. The system represents a modern cloud-native architecture combining synchronous and asynchronous processing.

This project will focus on performance evaluation. The system architecture will remain unchanged.

Objective

This study aims to answer:

1. How do response time and throughput change as concurrent user load increases?
2. Which services become performance bottlenecks under stress?
3. How does horizontal scaling impact latency and throughput?

Related Work

Microservice based architectures have become widely adopted for cloud applications due to their modular structure, independent deployability, and ability to scale individual components according to demand. Prior empirical evaluations of microservice deployments in cloud environments have shown that distributing functionality across multiple services introduces additional network communication and coordination overhead, which can increase request latency compared to local execution within a single process. These architectures rely on inter service communication over the network, making system performance sensitive to service interaction patterns and infrastructure configuration. Horizontal scaling is commonly used to improve throughput and handle increased load by replicating individual services, allowing resource allocation to be adjusted dynamically based on workload demands.

Many existing empirical studies focus on general purpose web applications and standard service interactions. There remains comparatively limited empirical performance characterization of compute intensive microservice systems that combine synchronous request response processing with asynchronous background workloads. This study aims to provide a reproducible workload based performance evaluation of such a system.

Reference: *M. Villamizar et al., "Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud," 2015 10th Computing Colombian Conference (10CCC), Bogota, Colombia, 2015, pp. 583-590, doi: 10.1109/ColumbianCC.2015.7333476. keywords: {Companies;Computer architecture;Cloud computing;Service-oriented architecture;Complexity theory;cloud computing;microservices;service oriented architectures;SOA;scalable applications;infrastructure as a services;platform as a service;PaaS;IaaS;continuous delivery;software engineering;software architecture;microservice architecture},*

Metrics

The primary metrics include:

- Mean, 95th, and 99th percentile response time*
- Throughput
- Error rate
- CPU and memory utilization per service
- RabbitMQ queue depth
- Redis cache hit rate

* Percentile latency is included to capture tail behavior under load.

Experimental Factors

The factors below will be varied. They allow isolation of scalability effects and bottleneck formation.

Factor	Levels
Concurrent Users	10, 50, 100, 200, 500
Service Replicas	1, 2,4
Cache	Enabled / Disabled
Request Type	Light API, AI-intensive
Workload Pattern	Constant, Spike, Stress

Evaluation Strategy and Setup

We will use a measurement-based evaluation approach. Backend services will be deployed in Docker containers. The load generator will run on a separate machine. System hardware and configuration will be documented for reproducibility.

Workloads will be generated using k6. A closed workload model will be used, where a fixed number of virtual users continuously send requests during each experiment.

Each test will include the following:

- 2-min warm-up
- 5-min steady-state measurement
- 1-min cool down

Load will increase incrementally from 10 to 500 concurrent users.

To ensure experimental validity and repeatability, the following variables will remain constant during testing:

- Hardware specifications (CPU cores, RAM)
- Operating system version
- Docker configuration
- Network environment
- Background system processes

Any configuration changes will be documented and version-controlled. This ensures that observed performance differences are attributable to experimental factors rather than environmental variation.

Workload Design

Three workload types will be tested:

1. Light API requests
2. AI-intensive workflows triggering asynchronous processing
3. Mixed workloads combining both

We will also perform:

1. Stress testing: we will gradually ramp-up until saturation
2. Spike testing: sudden traffic bursts

Workload distributions will be designed to approximate realistic user interaction patterns, including a mix of short-lived API calls and longer-running AI processing requests.

Bottleneck Identification

Bottlenecks will be identified by correlating latency increases with service-level resources utilization.

A service will be considered a bottleneck if:

- CPU utilization remains above 80% under load
- Memory usage approaches system limit
- Queue depth grows continuously
- Latency strongly correlates with service saturation

To validate bottlenecks, the suspected service will be scaled independently and overall performance changes will be measured.

Scaling Strategies

We will evaluate:

1. Horizontal scaling of all services (1, 2, 4 replicas)
2. Scaling only the identified bottleneck
3. Enabling / disabling Redis caching

Performance will be compared using throughput, percentile latency, and error rate.

Experiment Design and Analysis

Each experiment will be repeated 5 to 10 times to account for system variability. Measurements will be collected only during steady-state intervals to eliminate warm-up bias. For each metric, we will compute mean values and 95% confidence intervals. Statistical comparisons (t-tests) will be used to determine whether observed differences between configurations are statistically significant. Outliers will be documented and analyzed rather than automatically removed.

Experiments will be conducted in stages: baseline characterization, load scaling analysis, bottleneck validation, and scaling evaluation. This staged approach ensures progressive refinement of insights.

Timeline

Task	Deadline
Freeze experimental baseline and setup deployment & monitoring environment	March 6
Implement load generation scripts and collect baseline results	March 13
Midterm Progress Report	March 15
Conduct full load scaling experiments and bottleneck analysis	March 20
Perform scaling experiments and complete statistical analysis	April 6
Final Report	April 10